



Credit scorecards with **scorecraft** :: CHEAT SHEET

Points scorecards for binary targets (credit risk, fraud, propensity) on the optimal binning and WOE engine of **OptimalBinningWoE**, with explicit scale alignment, production SQL and IRB risk parameters.

The pipeline

EVERY STAGE IS A FUNCTION

Each stage returns an object that prints; every decision goes to a ledger. Two shortcuts chain the stages.

- 0 `scr_split()` type columns, split the sample
- 1 `scr_triage()` profile, sentinels, early failures
- 2 `scr_bin()` optimal bins and three gates
- 3 `scr_model()` model votes and consensus
- 4 `scr_scorecard()` logistic fit on WOE, points
- 5 `scr_align()` raw score to the declared scale
- 6 `scr_cutoff()` cut-off, strategy, rejects
- 7 `scr_sql()` scoring in R and in SQL

`scr_select()` = 0 to 3 `scr_scorecard()` = 4 and 5

Configure

`scr_config(preset = "moderate", ...)` Every knob of every stage in one object. The preset sets how tight the funnel is; override any key by name.

```
cfg <- scr_config("moderate", nthread = 4)
```

preset	variables	min votes	corr cut	IV floor
aggressive	10 to 15	3	0.6	0.03
moderate	10 to 25	2	0.7	0.02
lazy	10 to 40	1	0.8	0.02

`scr_config_keys` (stage = NULL) Every key with its stage (0 to 12), default and meaning.

`scr_presets()` This table. `scr_verbose`(on) Progress messages on or off.

Reading conventions

objective = "risk"
target 1 is bad; more points are safer (higher_is_safer, odds safe:event).

objective = "propensity"
target 1 is good; more points, more likely (higher_is_riskier, event:safe).

objective never changes the selection; event_level decides which target value is the event.

POINTS PER BIN: SC\$POINTS

vl_score_01	WOE	points
(-Inf; 33.36]	-2.06	61
(33.36; 38.15]	-0.73	22
(38.15; 44.24]	-0.66	20
(44.24; 48.06]	-0.52	16
(48.06; 63.94]	0.04	-1
(63.94; 72.61]	0.70	-21
(72.61; +Inf]	1.00	-30

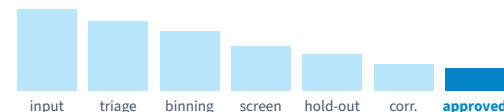
On `scr_demo`: score = 538 + points of 12 variables.

Select variables

STAGES 0 TO 3

```
res <- scr_select(scr_demo, "default", cfg,  
  drop = c("id", "churn"),  
  date_col = "ref_date")
```

`scr_select`(data, target, config, drop, date_col, event_level, export) Split, triage, binning and consensus in one call. With `date_col` the hold-out is out of time; without it, a stratified random 30%.



No candidate leaves the report: each one keeps the stage it failed at, and why.

STAGE BY STAGE

`scr_split`(data, target, date_col = NULL, ratio = 0.3) Type the columns and split train and hold-out.

`scr_triage`(split, config) Profile on train only. Fails CONSTANT, NEAR_CONSTANT, TOO_MANY_MISSING, HIGH_CARDINALITY, NO_SIGNAL, DUPLICATE_OF. A sentinel (-999) with mass and signal becomes a flag column `x__sp`.

`scr_bin`(triage, config) Optimal bins on train, in parallel by column, then three gates: the eight admission rules below; hold-out revalidation with **frozen bins** (IV ratio, PSI); redundancy pruning by rank correlation on the WOE space.

IV_BELOW_MIN IV_SUSPICIOUS NOT_MONOTONIC
TOO_FEW_BINS TOO_MANY_BINS SMALL_BIN DEGENERATE_BIN
BINNING_ERROR

`scr_model`(bins, config) **glmnet**, **xgboost**, **lightgbm** and **ranger** vote; the consensus is weighted by each model's hold-out Gini and the shortlist stays in [target_min, target_max].

Read the result

`scr_selected`(res, which = "final") The shortlist; also "consensus", "manual".

`scr_funnel`(res, only_selected = FALSE) Every input column, its IV, KS, PSI and the reason it stopped.

`scr_gains`(res) Bin-level gains of the approved variables.

`scr_leakage`(res, threshold = NULL) Suspicious IV and degenerate bins.

`scr_score_metrics`(sc) AUC, KS and Gini per sample, with bootstrap CI.

`scr_score_gains`(sc, sample = "holdout") Gains per score band frozen on train: KS, lift, pct_event, pct_nonevent and woe = ln(pct_event / pct_nonevent), > 0 when the band rate is above the overall rate; odds in the scale orientation.

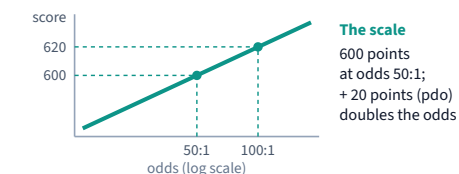
`summary`(res), `plot`(res), `as.data.frame`(res) Executive summary, funnel bar chart, funnel table.

Build the scorecard

STAGES 4 AND 5

```
sc <- scr_scorecard(res, base_score = 600,  
  base_odds = 50, pdo = 20,  
  challenger = "xgboost")  
sc$alignment # the fitted scale map
```

`scr_scorecard`(x, features, base_score, base_odds, pdo, direction, challenger, points_style) Logistic regression on the WOE columns with a **sign check** (a non-positive coefficient leaves, one at a time), points per bin, bootstrap CI, bands frozen on train, PSI and CSI.


$$\ln(\text{odds}) = I + S \cdot \text{logit} \rightarrow \text{score} = a + b \cdot \text{logit}$$
$$\text{factor} = \text{pdo} / \ln 2$$
$$\text{offset} = \text{base_score} - \text{factor} \cdot \ln(\text{base_odds})$$

`scr_align`(raw, y, base_score, base_odds, pdo, direction, method = "regression") Align the raw score of **any** engine: empirical log-odds regressed on score bands, composed with the PDO map. Two scorecards aligned this way compare point for point.

`predict`(align, raw, type = "score") Points, or type = "prob" for the implied probability.

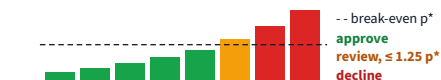
Challenger ("xgboost", "lightgbm"): aligned to the same scale for comparison, with `supports_scorecard = FALSE`: no points, no reason codes. **Points style**: "base_plus_deviation" or "distributed".

Decide

STAGE 6

`scr_cutoff`(sc, n_cuts = NULL, cuts = NULL) Approval, event rate on each side, events avoided and KS at each cut. Cuts are train quantiles applied frozen to the hold-out.

`scr_strategy`(sc, revenue_good = 1000, loss_bad = 4500) Bands with volume, event rate, odds_event, log_odds, decision and expected profit per account:

$$EP = (1 - p) \cdot \text{revenue_good} - p \cdot \text{loss_bad}$$
$$p^* = \text{revenue_good} / (\text{revenue_good} + \text{loss_bad})$$


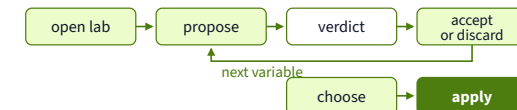
`scr_strategy`(sc, rule = "crossing") Cut where the event and non-event distributions cross (max KS, st\$crossing). Propensity: most likely band first; **target**, **review**, **skip**.

`scr_reject`(sc, population = NULL, accepted = NULL) Honest reject inference: population scope, outcome coverage per band and a **sensitivity band** (the rejects 2, 4 or 8 times worse), never a single invented multiplier.

Coarse classing lab

MANUAL BINS

Manual bins and manual variable choice, each with a reason, benchmarked against the optimal bins on train and hold-out.



Verdict: ACCEPTABLE, REVIEW or BLOCKED. The reason is mandatory and the ledger is append-only.

```
lab <- scr_coarse_classing(res)  
p <- scr_classing_propose(lab, "ds_region",  
  groups = list(edge = c("NORTH", "SOUTH"),  
    core = c("EAST", "WEST", "CENTRE")))  
lab <- scr_classing_accept(lab, p,  
  reason = "edge/core is what pricing uses")  
lab <- scr_classing_choose(lab,  
  drop = "vl_score_10",  
  reason = "not available at decision time")  
res2 <- scr_classing_apply(lab) # scr_result  
sc2 <- scr_scorecard(res2)
```

`scr_coarse_classing`(x, features = NULL, max_iv_loss = NULL) Open the lab on every variable that reached binning.

`scr_classing_view`(lab, variable) Current bins, train and hold-out IV.

`scr_classing_propose`(lab, variable, ...) One instruction per call:

breaks = c(20, 45) groups = list(...)
merge = c(2, 3) split = c(id, at) missing_to
other_to reset = TRUE

`scr_classing_accept`(lab, proposal, reason, override = FALSE) and `scr_classing_discard`(lab, proposal, reason) Record the decision; override accepts a BLOCKED one.

`scr_classing_choose`(lab, keep, drop, force, reason) The final variable list by hand.

`scr_classing_apply`(lab) Commit; the scorecard, the R scoring and the SQL follow unchanged.

`scr_classing_spec`(lab, file), `scr_classing_read`(file), `scr_classing_import`(lab, file) Specification round trip through CSV or xlsx, for a business reviewer.

`scr_decisions`(x) The decision ledger of a lab, result or scorecard.

Metrics on any vector

`scr_metrics`(score, y, higher_is_event = TRUE, n_boot = 200) AUC, KS and Gini with a bootstrap CI.

`scr_psi`(base, compare, n_groups = 10, alpha = 0.05) PSI with the fixed thresholds and the sample-size-adjusted critical value.

`scr_iv`(g, y, laplace = 0.5) Information value of any grouping.



Score in production

STAGE 7

```
scr_apply(sc, newdata) # score, points
scr_reasons(sc, newdata, k = 4)
scr_sql(sc, table = "prd.customers",
        dialect = "databricks",
        file = "score.sql")
scr_export(sc, "output")
```

scr_apply(sc, newdata, what = "score") The frozen pre-processing, bins and points; nothing is refitted. what: "score", "points", "woe", "all". On a selection: scr_apply(res, newdata, what = "both") gives WOE and bin labels.

scr_reasons(sc, newdata, k = 4, reference = "mean") Reason codes: the variables that took the most points from each row.

scr_sql(x, table, dialect, file) Production SQL in blocks: pre-processing CTE, WOE/BIN from the authoritative cut points, then the score (what = "score", "woe", "all"). **R and SQL agree**, verified by test.

ansi databricks spark hive mysql mariadb
sqlserver bigquery postgres oracle snowflake
redshift duckdb sqlite

scr_export(x, dir, stamp = TRUE) Deliverables in a timestamped folder: for a scorecard, scorecard_, validation_ and strategy_ workbooks plus the SQL; for a selection, the selection workbook, the WOE SQL and a Markdown summary.

Monitor

scr_monitor(sc, newdata, date_col, target) Per period: score PSI with frozen bands, CSI of every variable with the signed points shift and, with a target, AUC/KS/Gini by vintage.

scr_monitoring_plan(sc) The thresholds contract; edit the Monitoring_Plan sheet and pass it back as plan.



Next to it, the n-adjusted critical value: 0.034 at n = m = 1000 with 10 bins.

Database and many targets

```
con <- scr_connect(dsn = "DW")
rs <- scr_run(con, "dtm", config = cfg,
             targets = c("default", "churn"))
scr_compare(rs); scr_core(rs, min_targets = 2)
```

scr_connect(dsn, driver) ODBC with BIGINT read as numeric, or any DBI driver.

scr_fetch(con, table, sample_frac, seed) Reproducible server-side sampling.

scr_run(, scr_compare(), scr_core()) One selection per target, a comparison table, the variables that cross targets.

Parameters and default

scr_irb_params(framework) Editable tables: PD and LGD floors, supervisory LGD, CCFs, correlations, maturity, output floor, standardized weights.

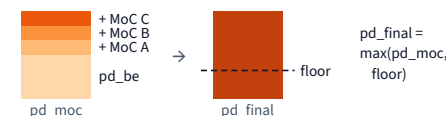
"bcb" "base13_final" "crr3"

scr_default(data, id, date, dpd, arrears, exposure, utp, restructured, obligor) Default flag from a monthly panel: 90 days past due with material arrears, or unlikelyness to pay; probation 3 months (12 if restructured); obligor pulling effect.

scr_default_rate(x, horizon = 12, by = "quarter") One-year default rates by cohort and the long-run average; lra_adjusted is benchmarked, never applied.

PD: calibration and grades

```
params <- scr_irb_params("bcb")
d <- scr_default(scr_demo_panel, id = "id",
               date = "ref_date", dpd = "dpd")
dr <- scr_default_rate(d, by = "quarter")
cal <- scr_calibrate(sc, target = dr)
gr <- scr_grades(sc, calibration = cal,
               n_grades = 8)
gr <- scr_moc(gr, "C", method = "ci_binomial")
pd <- scr_pd(gr, params = params,
            asset_class = "retail_other")
```



scr_calibrate(x, target, method) Re-anchor the PD to the central tendency; the points stay. Methods:

"intercept" "logodds_ab" "scaling" "qmm"

scr_master_scale(pd_min, pd_max, n_grades) Geometric master scale.

scr_grades(x, calibration, n_grades, method) Score cut points with monotone grade PDs ("geometric", "quantile", "supplied"); small grades merged and logged.

scr_moc(x, category, method, value, reason) Margin of conservatism: **C** estimation error is computed ("ci_timeseries", "ci_binomial", "bootstrap"); **A** and **B** need a value and a reason.

scr_pd(grades, params, asset_class, philosophy = "ttc") Final grade table with floor.

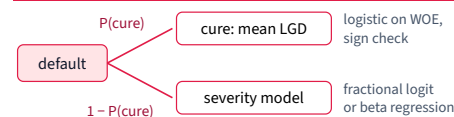
predict(pd, score = s, type = "pd_final") Grade, PD or final PD of new scores.

scr_pd_validate(x, newdata, id, date, default, score) Jeffreys, binomial, normal, Hosmer-Lemeshow, multi-period, AUC, concentration, PSI and migration, with traffic lights.

scr_migration(grade_t0, grade_t1) Migration matrix and bandwidths.

scr_pd_pit_ttc(pd, z, rho, to = "pit") One-factor PIT/TTC bridge.

LGD: workout and two stages



```
wo <- scr_workout(scr_demo_lgd,
                 scr_demo_lgd_cashflows,
                 rates = scr_demo_rates)
lgd <- scr_lgd(wo, drivers = c("product",
                              "ltv", "months_on_book"))
lgd <- scr_lgd_downturn(lgd, periods = dt,
                      reason = "rates above 13% in 2022-23")
lgd <- scr_lgd_floor(lgd, params = params)
# dt: data.frame(start, end) of downturn dates
```

scr_workout(defaults, cashflows, rates) Realized LGD per default: recoveries, direct costs and drawings discounted to the default date.

scr_lgd(x, drivers, holdout = 0.3) Cure x severity on cohort split, then pools.

scr_lgd_pools(x, n_pools) Re-pool the predicted LGD.

scr_lgd_downturn(x, periods, method, reason) Downturn per pool: "type1" observed impact, "type3" add-on, "none".

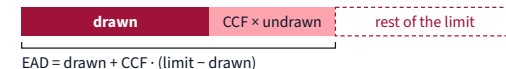
scr_lgd_floor(x, params, asset_class, secured_share) Input floors by collateral.

scr_lgd_validate(x) Calibration, discrimination (generalized AUC) and stability battery.

scr_elbe(x, grid = c(0, 6, 12, 24, 36)) ELBE and in-default LGD by months since default.

scr_bin_continuous(data, target, features) Monotone bins against a continuous target.

EAD: conversion factors



```
rds <- scr_ead_data(scr_demo_ead,
                  facility_id = "facility_id",
                  date_col = "ref_date", limit = "limit",
                  drawn = "drawn", defaulted = "defaulted",
                  drivers = c("product", "months_on_book"))
ead <- scr_ead(rds, drivers = c("product",
                              "utilisation_ref", "months_on_book"))
```

scr_ead_data(snapshots, facility_id, date_col, limit, drawn, defaulted, drivers) Realized CCF from monthly facility snapshots.

scr_ead(x, drivers, holdout = 0.3) Driver admission (TOO_FEW_DEFAULTS, NO_SEPARATION, NOT_MONOTONIC, UNSTABLE_HOLDOUT) and CCF pools with MoC and the standardized floor.

scr_ead_downturn(x, periods, method, reason) Downturn CCF per pool.

scr_ead_validate(x, newdata) Calibration, discrimination, back-testing, stability.

EL, capital and ECL

scr_el(pd, lgd, ead, defaulted, elbe) Expected loss per exposure, PD x LGD x EAD (ELBE when defaulted).

scr_irb_rw(pd, lgd, ead, m, asset_class, approach = "airb") IRB risk weight of the one-factor model, with floors, correlation and maturity adjustment.

$$K = LGD \cdot (scr_pd_stress(PD, R, 0.999) - PD) \cdot MA$$
$$RWA = 12.5 \cdot K \cdot EAD$$

corporate corporate_sme bank sovereign hvcre
retail_mortgage qrre-revolver qrre-transactor
retail_other

scr_sa_rw(asset_class, ltv, rating, ...) Standardized risk weight.

scr_pd_stress(pd, rho, q = 0.999) Conditional PD of the one-factor model.

```
cap <- scr_capital(scr_demo_portfolio,
                  segment = "segment",
                  asset_class = "asset_class",
                  provisions = "provision",
                  params = params)
cap$totals; cap$ssegments
```

scr_capital(x, pd, lgd, ead, segment, asset_class, provisions, params) RWA under IRB and the standardized approach, output floor, EL against provisions, floor impact, sensitivity and concentration.

scr_ecl(pd_term, lgd, ead, eir, stage, dpd, pd_orig, scenarios, weights) Expected credit loss from monthly hazards, discounted at the EIR, with weighted scenarios:

$$ECL = \sum_t S(t-1) \cdot h_t \cdot LGD_t \cdot EAD_t \cdot (1 + r)^{-t/12}$$

Stage 1	Stage 2	Stage 3
12-month ECL performing	lifetime ECL ≥ 30 dpd or 2 x PD	lifetime ECL ≥ 90 dpd, impaired

One contract for every model

object from	scr_apply	scr_sql	scr_export
scr_select()	●	●	●
scr_scorecard()	●	●	●
scr_coarse_classing()	—	—	●
scr_pd()	●	●	●
scr_lgd()	●	●	●
scr_ead()	●	●	●
scr_capital()	—	●	●

Demo data

SYNTHETIC

scr_demo	4,200 applications, two targets
scr_demo_panel	monthly panel for the default flag
scr_demo_lgd	defaults, _cashflows, scr_demo_rates
scr_demo_ead	monthly facility snapshots
scr_demo_portfolio	exposures for EL, capital and ECL



Score studies: bands, tiers, lights and operating points

One pass over the scored rows into counts per score value, then every study runs on the counts: **credit, fraud and propensity scores from any engine.**

Three ways in

```
scr_bands(sc) # a scorecard
scr_bands(df, score = "score", y = "y",
  objective = "propensity",
  sample = "sample",
  reference = "train") # any engine
scr_bands(agg, counts = TRUE) # score, n, events
```

A data.frame takes weight, value and a sample column; counts = TRUE reads a GROUP BY score done in the database.

Frozen cuts

fitted on the reference (train), midway between adjacent training scores; ties are never split.

One convention

score >= cut is the upper side; the event-richest band comes first.

Bands

PERCENTILES OR TAIL

`scr_bands(x, n_bands = 20, spacing = "uniform")` Per band and sample: rate with a Jeffreys interval, lift, capture, KS, WOE, IV, PSI and a Fisher test of rank order (`p_reversal_adj`, Holm). Per sample: AUC, Gini and KS with a bootstrap on the counts.

`spacing = "tail"` cuts the event-rich end at 0.1%, 0.5%, 1%, 2%, 5%, 10%, 20% and 50%: the fraud reading (alert rate, precision, recall).



band	rate [95%]	lift	capture	KS
1 [-Inf, 509.9)	35.4% [27.5, 44.0]	2.44	22.2%	0.153
2 [509.9, 523.5)	27.3% [20.2, 35.5]	1.89	39.4%	0.248
4 [533.4, 542.1)	17.4% [12.0, 24.1]	1.20	70.4%	0.370
10 [590.3, Inf)	3.2% [1.2, 6.8]	0.22	100%	0.000

`scr_bands(sc, n_bands = 10)` on the hold-out of `scr_demo`: AUC 0.739, KS 0.389, PSI 0.007 against train.

column	fraud and campaign reading
capture	recall at that depth
cum_rate	precision (hit rate) of the selection
cum_nonevent_pct	false positive rate
value_capture	share of the event value caught (value)

Large tables

The studies aggregate once (a keyed `data.table` pass) and then work on the distinct scores: 5 million rows take about one to three seconds.

`max_cells = 1e5` pools a continuous score into cells; `boot_cells = 1e4` bounds the cells of the bootstrap (exact below it).

`scr_export(study, dir)` One workbook per study.

Tiers

3, 5 OR 7 LABELS

```
tr <- scr_tiers(sc, n_tiers = 5, round_to = 5,
  n_boot = 100)
scr_apply(tr, newdata) # tier, tier_label
scr_sql(tr, table = "scored")
```

`scr_tiers(x, n_tiers = 5, method = "optimal")` Exact dynamic program on the training counts: maximizes the binomial likelihood (or the IV) with every tier above `min_pct` and `min_events`, monotone rates and adjacent tiers distinct (Fisher). An infeasible count falls back and is recorded in `ledger`.

`method = "anchored"` cuts at event-rate anchors, e.g. anchors = `c("overall", "0.6")`; `conservative = TRUE` uses the lower bound. `"quantile"`: equal shares.

01	02	03	04	05
very high	high	medium	low	very low

Labels are numbered from the highest event rate: "01.very high"; `scr_apply()` and `scr_sql()` take `numbered = FALSE` for plain labels. Not an IRB rating scale.

tier_label	score	share	rate [95%]
01.very high	< 500	5.4%	33.3% [23.5, 44.5]
02.high	[500, 515]	7.5%	37.1% [28.4, 46.6]
03.medium	[515, 540]	23.6%	23.0% [18.7, 27.8]
04.low	[540, 555]	19.6%	11.6% [8.3, 15.8]
05.very low	≥ 555	43.9%	5.0% [3.5, 7.0]

Fitted on train, read on the hold-out: there the two top tiers are not distinct (p 0.75), a reason to use four. Across 100 resamples 83% of the rows keep their tier.

```
CASE ... WHEN s.score < 500 THEN '01.very high'
... ELSE '05.very low' END AS tier_label
```

Lights

RED, AMBER, GREEN

`scr_rag(x, plan = NULL, by = NULL)` Discrimination (Gini ratio, AUC change), calibration (O/E, bands), stability (PSI, rank order) and, for a scorecard, the variables (CSI, IV ratio, WOE sign).

`green` `amber` `red` `grey: too few events`

`scr_rag_plan(objective)` The editable thresholds. A light turns amber or red only when the confidence interval shows the deviation; calibration is one-sided under risk, two-sided under propensity.

check	green	red
gini_ratio	≥ 0.95	< 0.90
auc_change_p	> 0.05	≤ 0.01
oe_ratio	≤ 1.10	> 1.25
score_psi, csi	< 0.10	≥ 0.25, significant
rank_order	0 reversals	2 or more
iv_ratio	≥ 0.80	< 0.50

Defaults under risk. Propensity: Gini ratio 0.90 and 0.80; O/E green within 0.90 to 1.10, red outside 0.80 to 1.25.

Claims

TESTED STATEMENTS

```
cl <- data.frame(score_lo = 500, op = ">=",
  rate = c(0.60, 0.70))
scr_claims(sc_prop, cl)
```

`scr_claims(x, claims, level = 0.95, type = "average")` One row per claim, on a band or tier (`label`) or a score range: exact one-sided binomial test, Jeffreys bound, Holm. The verdict is **supported**, **refuted** or **not proven**, with the sentence written out. `type = "floor"` tests the weakest end of the group.

group	n	rate	bound	claim	verdict
score >= 500	104	73.1%	65.5%	≥ 60%	supported
score >= 500	104	73.1%	65.5%	≥ 70%	not proven

"On 'holdout' (n = 104), rows with score >= 500 had an event rate of 73.1% (95% one-sided lower bound 65.5%); the claim 'rate >= 60%' is supported."

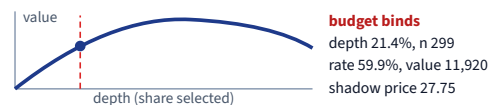
Operating point

```
scr_operating(sc_prop, gain_event = 100,
  cost_select = 20, budget = 6000)
```

`scr_operating(x, side = NULL, ...)` The curve from the event-rich end ("event": targeting, alerting) or from the safe end ("safe": approval with `revenue_good`, `loss_bad`), the optimum, the binding constraint and its shadow price.

`max_n` `max_share` `budget` `max_per_day` `min_rate` `max_rate`

`max_per_day` reads a quantile of the daily volume (`day_quantile = 0.9`): review capacity.



The shadow price is what one more selected case is worth at the binding constraint: the number to argue for budget or staff.

```
scr_operating(sc, revenue_good = 1080,
  loss_bad = 4500, max_rate = 0.10) # approval
```

Two scores

`scr_score_cross(df, score_a, score_b, y_a, y_b)` Band by band cross table with the rate of one or two outcomes, Spearman and Kendall tau-b, and the overlap of the two selected lists at `depths`, with swap-in and swap-out rates.

```
scr_score_cross(df, "score_a", "score_b",
  y_a = "default", y_b = "churn",
  objective_a = "risk",
  objective_b = "propensity")
```

Cuts may come from a tiers study (`cuts_a = tr`), which also sets the direction of that score.

What changed, and where

`scr_mix_shift(x, by = NULL)` Splits the change of the event rate into a **mix** effect (the population moved across bands) and a **rate** effect (the bands deteriorated); the two sum exactly to the change.

2026-06: 14.1% → 15.0% (+0.86 pp)
= mix -1.04 pp + rate +1.90 pp

With `by = "date"`, `n_bands = 10`: June against January.

`scr_segments(sc, newdata, segment)` One score per segment: AUC, O/E, offset, slope ratio and PSI, tests of equal AUC and slope, and an action.

`shared` `offset` `separate model` `too few events`

segment	AUC	O/E	slope ratio	action
APP	0.753	1.02	0.93	shared
STORE	0.788	1.02	1.11	shared
WEB	0.778	0.94	1.07	shared

`scr_demo` by channel: equal AUC is not rejected (p 0.35).

Time and treatment

`scr_maturity(df, time, event, horizons)` Kaplan-Meier cumulative incidence per band and horizon with censoring: where the curve flattens is the outcome window. `time` runs from the origin to the event or to the end of follow-up; `event` is 1 when it was observed.

`scr_uplift(df, treat = "treat")` Treated against control per band (Newcombe interval), Qini and AUUC with a bootstrap, and a check that the assignment was random.

`persuadable` `no effect` `negative`

A high propensity is not a high uplift.

Fraud operations

`scr_overlap(df, rules, alert_share)` Rules against the score: overlap sets, incremental recall in counts and in value, and the rules the score already covers.

`scr_detection(df, entity, time, alert_shares)` Per threshold: episodes detected, event rows and time before detection, and the loss prevented.

Which study answers what

Does the score rank?	scr_bands
Which labels for the business?	scr_tiers
Is the model still healthy?	scr_rag
Can we state "above 60%"?	scr_claims
Where to cut, given capacity?	scr_operating
Why did the rate move?	scr_mix_shift
One model or one per segment?	scr_segments
Do two scores pick the same rows?	scr_score_cross
When is the outcome mature?	scr_maturity
Who responds because of us?	scr_uplift
Which rules can retire?	scr_overlap
How fast is fraud caught?	scr_detection